

AADK 8: Task 3 — Implementation Reflection & Test Coverage Audit

Student Name: Rajat Prakash Dhal

Course/Unit: Android Development with Kotlin – Session 8

USN: 1hk22cs115

email: 1hk22cs115@hkbk.edu.in

1. Implementation (Minimum Viable)

In this task, two main features from the Art Space app were implemented and tested:

1. **Next Artwork Button Behavior**
2. **Favourite Toggle Feature**

The **Next button** allows the user to move to the next artwork in the gallery. When clicked, the application updates the `currentArtworkIndex` state variable and displays the next artwork image along with its title and artist information.

The **Favourite toggle feature** allows users to mark an artwork as a favourite by clicking a heart icon. When the icon is clicked, the artwork is either added to or removed from the favourites list depending on its previous state.

Example Business Logic (Unit Test Target)

```
fun getNextIndex(currentIndex: Int, total: Int): Int {  
    return if (currentIndex < total - 1) currentIndex + 1 else 0  
}
```

This function ensures that when the last artwork is reached, the index loops back to the first artwork.

Example Unit Test

```
@Test  
fun nextIndex_returnsCorrectValue() {
```

```
    val result = getNextIndex(0, 5)

    assertEquals(1, result)
}
```

This unit test verifies that the function correctly increments the artwork index.

Example Compose UI Test

@get:Rule

```
val composeTestRule = createComposeRule()
```

@Test

```
fun nextButton_updatesArtwork() {
    composeTestRule.setContent {
        ArtSpaceApp()
    }
}
```

```
composeTestRule
```

```
    .onNodeWithText("Next")
```

```
    .performClick()
```

```
composeTestRule
```

```
    .onNodeWithText("Artwork 2")
```

```
    .assertExists()
```

```
}
```

This UI test simulates a user clicking the **Next button** and verifies that the next artwork title appears on the screen.

2. Test Coverage Audit

Component	Has Test?	Type of Test	Coverage Status	Priority
Next Button Navigation	Yes	UI Test	Good	High
Favourite Toggle Logic	Yes	Unit Test	Good	High
Artwork Details Screen	No	UI Test Needed	Needs Improvement	Medium
Orientation Change Handling	No	UI Test Needed	Needs Improvement	High
Favourites Persistence	No	Unit + UI Test Needed	Needs Improvement	Medium

Comments

- **Next Button Navigation:** Fully tested with UI interaction test to ensure artwork updates correctly.
- **Favourite Toggle Logic:** Business logic tested using unit tests to confirm correct state changes.
- **Artwork Details Screen:** Currently implemented but lacks automated UI tests to verify data display.
- **Orientation Change Handling:** State preservation during device rotation has not been tested.
- **Favourites Persistence:** Saving favourites across app restarts requires additional tests.

3. Bug / Edge-Case Discovery

One edge case discovered during testing is related to **device rotation**. When the device rotates, the `currentArtworkIndex` may reset to the default value instead of preserving the current artwork state.

To address this issue, the app should use a **ViewModel or saved state handling** to preserve the state during configuration changes.

A new UI test could simulate orientation changes and verify that the currently displayed artwork remains the same after rotation.

4. Reflection

Writing tests during development helped ensure that the UI features behaved as expected and reduced the chances of introducing bugs later. It also encouraged structuring the code in a more modular and testable way. One challenge faced was setting up UI tests that correctly locate UI elements in the Compose semantics tree. Another issue was managing state changes during tests. In the future, I would prioritize separating business logic from UI components and using ViewModels to make testing easier and more reliable.